

Incident RCA: Tasker High CPU Exhaustion (INC-002)

Metadata	Details
Incident ID	INC-002
Service	Tasker Web API (<code>tasker-tasker-chart</code>)
Environment	Kubernetes (Namespace: <code>default</code>)
Severity	Warning
Status	Resolved
Root Cause	Unbounded CPU stress process manually injected into the pod container
Detection Alert	<code>TaskerHighCPU</code> (Prometheus)
Simulation Script	<code>tasker/tests/simulate-high-cpu.sh</code>

1. Executive Summary

To validate Prometheus alerting under sustained CPU pressure without an application crash, a bounded busy-loop (`timeout 400 sh -c "while true; do :; done"`) was executed directly inside the running Tasker container. The container has a CPU request of `100m` and a limit of `250m`; the `TaskerHighCPU` alert is configured to fire when CPU usage exceeds 80% of the request (`80m`) for a sustained 5-minute window.

CPU utilization rose to approximately the container’s cgroup limit (~`250m`) while the Tasker web process remained `1/1` Running with 0 restarts throughout – the stress load did not affect application availability. `TaskerHighCPU` transitioned to firing after the sustained 5-minute evaluation window. Because the container’s base image does not bundle a standalone `kill/pkill` binary, the stress process was terminated from the host by entering the container’s PID namespace via `nsenter`, targeting only the stress-loop PIDs and leaving the Unicorn master/worker processes untouched. CPU usage returned to baseline (~`1m`) immediately after termination, and the alert cleared.

Incident in one sentence: We intentionally ran a CPU-burning process inside the Tasker container, driving CPU usage to roughly the pod’s `250m` limit against an `80m` alert threshold; Prometheus detected the sustained high CPU after ~5 minutes, we identified the stress process via the container’s process list, terminated only that process from the host without restarting Tasker, CPU returned to normal, and the alert cleared.

2. Resource Allocation & Threshold Architecture

Defined in Helm `values.yaml`:

```
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 250m
    memory: 256Mi
```

- **Base request:** `100m` (0.10 vCPU)
- **Alert threshold:** 80% of request = `80m`
- **Observed peak usage:** ~`250m` (at the container’s hard limit)

3. Incident Timeline

Note: Exact timestamps for this incident were not captured at the time and could not be reconstructed afterward – Kubernetes does not retain CPU-threshold events long-term, and the pod involved has since been replaced by later deployment restarts (unrelated to this incident). The sequence and outcome below are accurate; clock times are omitted rather than presented as precise when they are not.

Phase	Action / Event
Trigger	<code>simulate-high-cpu.sh</code> executed, running a bounded CPU busy-loop inside the Tasker container via <code>kubectl exec</code> .
Impact	Tasker container CPU usage rose from near-idle (~ <code>1m</code>) to approximately its cgroup limit (~ <code>250m</code>).
Sustained load	Elevated CPU persisted continuously for the alert’s required 5-minute evaluation window.
Detection	<code>TaskerHighCPU</code> transitioned to FIRING (severity: warning)

	once the 5-minute sustained condition was met.
Investigation	Confirmed via <code>kubectl get pods</code> and <code>kubectl top pod</code> that Tasker remained <code>1/1 Running</code> with 0 restarts – the load was degrading available compute, not crashing the application. Inspected the container’s process list and identified the stress loop as a separate process tree from the Gunicorn master/worker.
Mitigation	Located the container’s host PID via the node’s container runtime (<code>crictl</code>), then used <code>nsenter</code> to enter its PID namespace and send <code>SIGTERM</code> to the stress-loop PIDs specifically – avoiding Gunicorn PIDs 1 and 7.
Recovery	CPU usage dropped back to near-idle (~1m) immediately; pod remained <code>1/1 Running</code> , 0 restarts throughout.
Resolution	<code>TaskerHighCPU</code> cleared back to <code>INACTIVE</code> once CPU stayed below threshold.

4. Trigger & Execution Mechanism

```
kubectl exec -n default deploy/tasker-tasker-chart -- \
  sh -c 'timeout 400 sh -c "while true; do ;; done"'
```

- The `while true; do ;; done` construct is a zero-I/O busy loop that pegs a CPU thread at 100% utilization.
- The `timeout 400` wrapper bounds the stress process to a maximum of 400 seconds as a safety net, in case manual termination failed.
- The Linux cgroup CPU controller throttled the container at its configured limit (`250m` / 0.25 cores) rather than allowing unbounded consumption.
- Gunicorn’s worker processes run independently of the shell loop, so HTTP-serving capacity was degraded (less CPU share available) but the application did not crash or exit.

5. Detection & Alerting Verification

```
- alert: TaskerHighCPU
  expr: (
    rate(container_cpu_usage_seconds_total{container="tasker", image!=""}[2m])
    /
    kube_pod_container_resource_requests{resource="cpu", container="tasker"}
  ) * 100 > 80
for: 5m
labels:
  severity: warning
annotations:
  summary: "Tasker high CPU utilization detected"
  description: "Tasker container CPU usage is {{ $value | humanizePercentage }} (above 80% request threshold) for 5 minutes."
```

(Verify this against the actual `PrometheusRule` manifest in the repo before finalizing – copy it in verbatim to guarantee an exact match.)

Alert firing and clearing were confirmed via the Alertmanager/Grafana alert state view. `kubectl top pod -n default` during the stress window showed Tasker’s CPU usage well above the PostgreSQL pod’s baseline, consistent with the stress process running only inside the Tasker container.

6. Investigation & Process Identification

```
kubectl get pods -n default
```

Confirmed Tasker remained `1/1 Running` with 0 restarts throughout – unlike Incident 1, no crash or restart occurred; this was a pure resource-pressure event.

Process inspection inside the container distinguished two process groups:

- **Gunicorn master/worker** (PIDs 1 and 7 in that run) – legitimate application processes, left untouched.
- **The injected stress loop** (`sh -c timeout 400 sh -c "while true; do ;; done"` and its child shell) – the source of the CPU load.

7. Recovery & Surgical Mitigation

Complication: The container’s base image does not include standalone `kill/pkill` binaries, so the stress process could not be terminated from inside the container via `kubectl exec`.

Resolution procedure:

1. Identified the container's ID on the node via the container runtime (`crictl ps`, filtered to the Tasker pod).
2. Retrieved the corresponding host PID for that container (`crictl inspect`).
3. Entered the container's PID namespace from the host and sent `SIGTERM` specifically to the stress-loop PIDs:

```
sudo nsenter -t 71259 -p -- /usr/bin/kill -TERM 15 16
```

Gunicorn's master (PID 1) and worker (PID 7) processes were deliberately excluded from this signal to avoid restarting or degrading the running application.

Verification of impact:

- CPU usage dropped from ~250m to ~1m immediately after termination.
- Pod state remained `1/1 Running` with `0` restarts throughout the entire incident.
- Prometheus/Alertmanager confirmed `TaskerHighCPU` returned to `INACTIVE` once CPU stayed below the 80m threshold.

8. Root Cause Analysis (5 Whys)

1. **Why did Prometheus raise the `TaskerHighCPU` alert?** CPU utilization exceeded 80m (80% of the 100m request) continuously for over 5 minutes.
2. **Why was CPU utilization elevated?** A non-yielding shell busy-loop was actively executing inside the container.
3. **Why was the loop running?** It was deliberately injected via `kubect1 exec` as part of this controlled failure simulation, to validate CPU-pressure alerting.
4. **Why did the application continue serving despite the CPU spike?** The container's CPU limit throttled the stress process at the cgroup level without killing it, and Gunicorn's worker processes retained a share of available CPU cycles independently of the stress loop.
5. **Why was `nsenter` required for resolution rather than a simple `kubect1 exec kill`?** The production container image intentionally omits general-purpose debugging utilities (`kill`, `pkill`), requiring host-level process-namespaces access for targeted, zero-downtime termination.

Root cause: A deliberately injected CPU-bound process consumed available container compute up to its configured limit; this is the expected/intended outcome of the simulation and validates that Prometheus alerting correctly detects sustained resource pressure that does not otherwise crash the application.

9. Preventative & Detective Action Items

Category	Action Item	Target	Owner
Detective	Keep <code>TaskerHighCPU</code> threshold at 80% of request with a 5m window - this correctly caught sustained pressure without false-alarming on short bursts.	Prometheus	SRE
Architectural	Evaluate a Horizontal Pod Autoscaler (HPA) targeting ~80% CPU utilization to automatically scale under genuine sustained load.	Kubernetes	DevOps
Security	Restrict <code>kubect1 exec</code> RBAC privileges in production to prevent unauthorized process injection via this same mechanism.	RBAC / Cluster	SecOps
Operational	Document the <code>nsenter/crictl</code> procedure in the runbook for zero-downtime termination of rogue processes when a minimal container image lacks debugging tools.	Runbooks	SRE